

Лабораторная №3: *Abstract Factory*

Задание 1

1. Разработайте библиотеку классов **Lec03LibN**, реализующую паттерн **Abstract Factory**.
2. Библиотека позволяет вычислить величину вознаграждение сотрудника.
3. Величина вознаграждение сотрудника зависит от отработанного времени (часы), стоимости 1 часа и вычисляется в зависимости от типа и уровня вознаграждения. Формулы для вычисления вознаграждения сведены в следующей таблице.

	Типы			
		A ()	B (x)	C (x, y)
Уровни	1 ()	$wH^1 * cH^2$	$wH * cH * x^3$	$wH * cH * x + y^4$
	2 (a)	$(wH + a^5) * cH$	$(wH + a) * cH * x$	$(wH + a) * cH * x + y$
	3 (a, b)	$(wH + a) * (cH + b^6)$	$(wH + a) * (cH + b) * x$	$(wH + a) * (cH + b) * x + y$

- 1 – отработанные часы;
- 2 – стоимость 1 часа;
- 3 – повышающий/понижающий коэффициент;
- 4 – величина повышения/понижения;
- 5 – величина повышения/понижения отработанных часов;
- 6 – величина повышения/понижения стоимости 1 часа.

4. Библиотека классов **Lec03LibN** предоставляет пользователю (программисту) только:

```
public interface IBonus           // вознаграждение
{
    float cost1hour { get; set;}   // стоимость одного часа
    float calc(                   // вычисление вознаграждения
        float number_hours       // - количество отработанных часов
    );
}
```

```

public interface IFactory                                // типы вознаграждения
{
    IBonus getA(                                         // тип вознаграждения A
        float cost1hour                                // стоимость 1 часа
    );
    IBonus getB(                                         // тип вознаграждения B
        float cost1hour,                                // стоимость 1 часа
        float x                                          // повышающий/понижающий коэффициент
    );
    IBonus getC(                                         // тип вознаграждения C
        float cost1hour,                                // стоимость 1 часа
        float x,                                         // повышающий/понижающий коэффициент
        float y                                          // величина снижения/повышения
    );
}

```

```

static public partial class Lec03BLib                    // уровни вознаграждения
{
    static public partial IFactory getL1();              // уровень 1

    static public partial IFactory getL2(               // уровень 2
        float a                                         // величина снижения/повышения отработанных часов
    );
    static public partial IFactory getL3(               // уровень 3
        float a,                                         // величина снижения/повышения отработанных часов
        float b                                         // величина снижения/повышения стоимости 1 часа
    );
}

```

ВНИМАНИЕ! Наименования интерфейсов и класса принципиально должны быть такими же.

5. Предполагается, что сгенерированный библиотекой (точнее создаваемые методами класса **Lec03BLib** реализации абстрактной фабрики **IFactory**) объект, реализующий интерфейс **IBonus**, может быть применен в соответствии с принципом **Dependency Inversion Principle** (один из принципов **SOLID**). Ниже приведен пример класса **Employee**, в котором внедряется алгоритм вычисления величины вознаграждения через параметр конструктора.

```

public class Employee                                    // сотрудник, получающий вознаграждение
{
    public IBonus bonus { get; private set; }           // алгоритм для вычисления вознаграждения
    public Employee(IBonus bonus)                       // Dependency Inversion Principle
    {
        this.bonus = bonus;
    }
    public float calcBonus(float number_hours)          // вычисление вознаграждения
    {
        return bonus.calc(number_hours);               // вызов внедренного алгоритма
    }
}

```

6. Разработайте приложение **PP03**, применяющее библиотеку **Lec03LibN** и выполняющее следующий тест.

```
using Lec03BLib; // библиотека
using PP03;      // сборка приложения
class Program
{
    static void Main(string[] args)
    {
        Console.WriteLine("Лабораторная работа # 3");

        IFactory l1 = Lec03BLib.getL1(); // фабрика 1

        Employee employee1 = new Employee(l1.getA(25)); // 1-a
        Console.WriteLine(string.Format("Bonus-L1-A = {0}", employee1.calcBonus(4)));

        Employee employee2 = new Employee(l1.getB(25, 1.1f)); // 1-b
        Console.WriteLine(string.Format("Bonus-L1-B = {0}", employee2.calcBonus(4)));

        Employee employee3 = new Employee(l1.getC(25, 1.1f, 5.0f)); // 1-b
        Console.WriteLine(string.Format("Bonus-L1-C = {0}", employee3.calcBonus(4)));

        IFactory l2 = Lec03BLib.getL2(1); // фабрика 2

        Employee employee4 = new Employee(l2.getA(25)); // 2-a
        Console.WriteLine(string.Format("Bonus-L2-A = {0}", employee4.calcBonus(4)));

        Employee employee5 = new Employee(l2.getB(25, 1.1f)); // 2-b
        Console.WriteLine(string.Format("Bonus-L2-B = {0}", employee5.calcBonus(4)));

        Employee employee6 = new Employee(l2.getC(25, 1.1f, 5.0f)); // 2-c
        Console.WriteLine(string.Format("Bonus-L2-C = {0}", employee6.calcBonus(4)));

        IFactory l3 = Lec03BLib.getL3(1, 0.5f); // фабрика 3

        Employee employee7 = new Employee(l3.getA(25)); // 3-a
        Console.WriteLine(string.Format("Bonus-L3-A = {0}", employee7.calcBonus(4)));

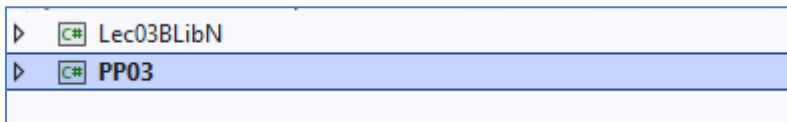
        Employee employee8 = new Employee(l3.getB(25, 1.1f)); // 3-b
        Console.WriteLine(string.Format("Bonus-L3-B = {0}", employee8.calcBonus(4)));

        Employee employee9 = new Employee(l3.getC(25, 1.1f, 0.5f)); // 3-c
        Console.WriteLine(string.Format("Bonus-L3-C = {0}", employee9.calcBonus(4)));
    }
}
```

Консоль отладки Microsoft Visual Studio

```
Лабораторная работа # 3
Bonus-L1-A = 100
Bonus-L1-B = 110
Bonus-L1-C = 115
Bonus-L2-A = 125
Bonus-L2-B = 137,5
Bonus-L2-C = 142,5
Bonus-L3-A = 127,5
Bonus-L3-B = 140,25
Bonus-L3-C = 138
```

7. Библиотека **Lec03LibN** и приложение **PP03** должны быть разработаны в рамках общего решения Visual Studio.



ВНИМАНИЕ! Наименования проектов принципиально должны быть такими же.

8. Убедитесь, что тест выполняется корректно.

Задание 2

9. Разработайте детальную UML-диаграмму реализации библиотеки **Lec03LibN** и класса **Employee**.
10. Поясните все обозначения UML-диаграммы.

Задание 3. Ответы на вопросы

11. Перечислите и поясните принципы **SOLID**.
12. Поясните детально, каким образом **Dependency Inversion Principle** применен в реализации класса **Employee**.
13. Поясните суть паттерна **Abstract Factory**, что дает его применение.
14. Сравните два паттерна **Factory Method** и **Abstract Factory**.
15. Как (какие классы, методы и как) необходимо доработать библиотеку **Lec03LibN**:
 - если появится новый тип вознаграждения $D(x, y, z) = (wH + a) * cH * x + y * z$, действующий, только для уровня 2;
 - если появится новый уровень вознаграждения $4(a, b, c) = (wH + a * c) * (cH + b)$, действующий, только для уровня 1;
 - если перестанет применяться уровень вознаграждения типа B уровня 3.